

LINGUAGEM DE PROGRAMAÇÃO C++

Curso de Capacitação em Desenvolvimento Full Stack - MÓDULO 08 (40H)

Elaborado por: Prof(a). Esp. Érika D. G. R. Dos Santos

PROGRAMAÇÃO ORIENTADA A OBJETOS

AULA 9: LINGUAGEM DE PROGRAMAÇÃO C++

Indtrodução a POO

SUMÁRIO

- 1 - Conceito Básico
- 2 - Paradigma POO vs. Procedural
- 3 - Componentes principais: classe, objeto, atributo, método
- 4 - Exercício de fixação

O que é Programação Orientada a Objeto ou POO ?



Conceitos Básico

O mundo é visto como sendo composto por objetos, onde um objeto é uma entidade que combina estrutura de dados e comportamento funcional.



Paradigmas

P00 x Procedural

PROCEDURAL

**Dados e
funções**



funções



P00

Objetos



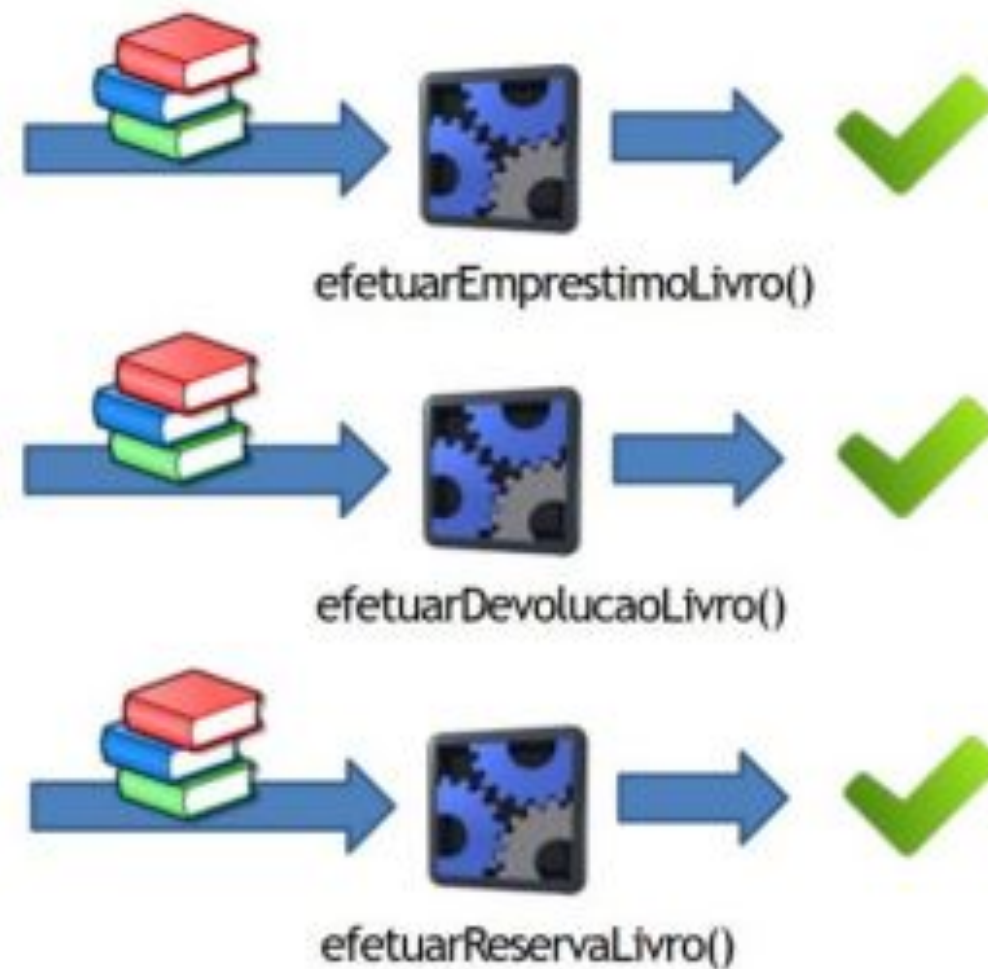
dados



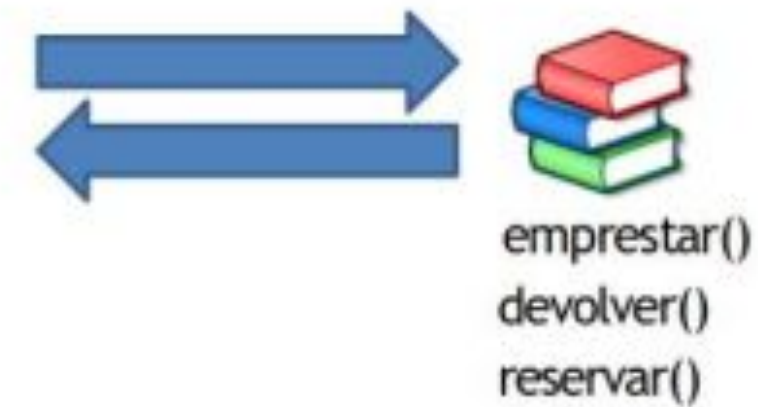
métodos

Paradigmas

Procedural x P00



Procedural



Diferentes visões de mundo!!!

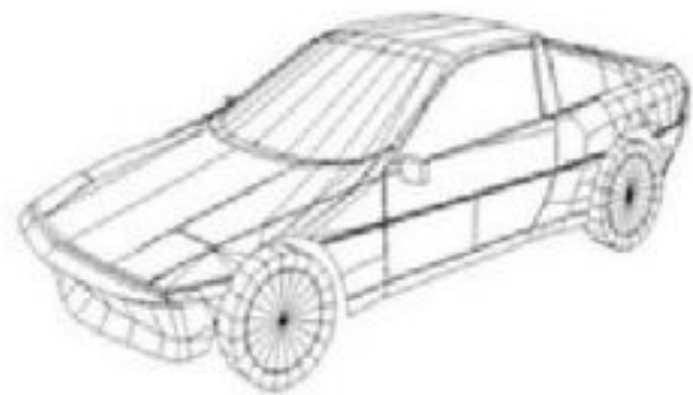
Orientado a Objeto

Componentes principais

Classe, Objeto, Atributo e Método

O que é Classe ?

Uma Classe é um molde ou projeto para se criar “objetos” daquela classe.



Componentes principais

Classe, Objeto, Atributo e Método

O que é Objeto ?

Um Objeto é uma instância de uma Classe, que define um tipo e suas características (atributos) e comportamentos (métodos).



Tipo: Ferrari
Cor: Vermelho
Placa: JKL-0001
Número de Portas: 4



Tipo: Casa
Piso: Cerâmica
Quartos: 3
Telhado: Telha de barro
Banheiro: 2

Componentes principais

Classe, Objeto, Atributo e Método

Característica (atributos, em OO)

São elementos que definem informações ou propriedades que descrevem o estado de algo, podendo ser atribuídos a variáveis, objetos ou classes que é composto por um tipo de dado.

Atributos de “Carro”: cor, motor, ano, marca, etc.

Comportamento (métodos, em OO)

Ações que podem ser realizadas pelo objeto

Comportamentos de um “Carro”: ligar, acelerar, diminuir a velocidade, parar, etc.



🚗 Carro	
🚦 ATRIBUTOS (CARACTERÍSTICAS)	
• tipo:	?
• cor:	?
• placa:	?
• numeroPortas:	?
• velocidade:	?
⚡ MÉTODOS (COMPORTAMENTOS)	
• ligar():	?
• acelerar():	?
• frear():	?
• getVelocidade():	?

Componentes principais

Atributo e Tipos de Dados (Primitivos)

Tipos Primitivos (Simples):

Usados para representar valores básicos:

- int: números inteiros
- float: números decimais (precisão simples)
- double: números decimais (precisão dupla)
- char: caractere único
- bool: valores lógicos (True ou False)
- str: textos ou cadeias de caracteres
- void: ausência de tipo
- nullptr: ponteiro nulo (C++11)



Carro

ATRIBUTOS

- tipo: `std::string`
- cor: `std::string`
- placa: `std::string`
- numeroPortas: `int`
- velocidade: `int`

MÉTODOS

- + ligar(): `void`
- + acelerar(): `void`
- + frear(): `void`
- + getVelocidade(): `int`

Componentes principais

Atributo e Tipos de Dados (Compostos - (STL))

Tipos Compostos (Estruturados):

Usados para agrupar múltiplos valores:

- `std::vector`: lista dinâmica
- `std::array`: array de tamanho fixo
- `std::map`: dicionário chave-valor
- `std::unordered_map`: hash map (mais rápido)
- `std::set`: conjunto de valores únicos
- `std::tuple`: tupla (C++11)
- `std::string`: cadeia de caracteres



Tipos Compostos (C++)

Tipos Compostos (Estruturados)

<code>std::vector</code>	:	?	ex: {1, 2, 3}
<code>std::map</code>	:	?	ex: {"nome": "João"}
<code>std::set</code>	:	?	ex: {1, 2, 3}
<code>std::tuple</code>	:	?	ex: (1, "Oi", 3.14)
<code>std::string</code>	:	?	ex: "texto"

Métodos Comuns

<code>vector.push_back()</code>	:	?
<code>map.find()</code>	:	?
<code>set.insert()</code>	:	?

Complete os (?) com a descrição correta

Componentes principais

Função vs Método

- **Função:**

- Bloco de código independente que realiza uma tarefa.
- É um bloco de código independente.
- Não está ligada a nenhuma classe ou objeto.
- Pode ser chamada a qualquer momento no programa

```
int somar(int a, int b) {  
    return a + b; // função normal (global)  
}  
  
int main() {  
    cout << somar(3, 5);  
    return 0;  
}
```



Componentes principais

Função vs Método

- **Método:**

- É uma função definida dentro de uma classe.
- Está associada a um objeto ou à própria classe.
- Para ser usada, precisa precisa de um objeto (ou da classe para métodos estáticos).
- O primeiro parâmetro de um método de instância em C++ normalmente é **this**, que representa o objeto que está chamando o método. Ponteiro this (implícito, não é parâmetro explícito)

```
class Calculadora {  
public:  
    int somar(int a, int b) {  
        return a + b; // this está implícito  
    }  
};  
  
int main() {  
    Calculadora calc;  
    cout << calc.somar(2, 3); // método  
    return 0;  
}
```



Componentes principais

Classe, Objeto, Atributo e Método

Características (atributos)

Tipo: Ferrari

Cor: Vermelho

Placa: JKL-0001

Número de Portas: 4

Ações (métodos)

Ligar()

Acelerar()

Frear()

...



Carro



Componentes principais

Classe, Objeto, Atributo e Método



Casa

Características (atributos)

Tipo: Casa

Piso: Cerâmica

Quartos: 3

Telhado: Telha de barro

Banheiro: 2

Ações (métodos)

abrirPorta()

acenderLuzes()

fecharJanela()

...



Representação de uma classe

O que é o Construtor?

O **construtor** é um método especial da classe em C++ que é **executado automaticamente** quando criamos um novo objeto. Ele serve para **inicializar os atributos** da instância e alocar recursos necessários.

E o que é this?

- **this** é um **ponteiro para o próprio objeto** atual
- Sempre está disponível **dentro dos métodos não estáticos** da classe
- É o **this** que permite que cada objeto acesse **suas próprias informações** (seus atributos)
- Usamos **this->atributo** ou **(*this).atributo** para acessar membros

```
class Cachorro {  
private:  
    std::string nome;  
  
public:  
    // Construtor usando this para resolver ambiguidade  
    Cachorro(std::string nome) {  
        this->nome = nome; // this->nome é o atributo do objeto  
    }  
  
    void latir() {  
        std::cout << this->nome << " diz: Au Au!" << std::endl;  
    }  
};  
  
Cachorro c1("Rex");  
Cachorro c2("Bob");  
  
// Cada objeto tem seu próprio 'this' com valores diferentes  
c1.latir(); // Rex diz: Au Au!  
c2.latir(); // Bob diz: Au Au!
```

Vamos praticar?



Exercício de fixação

**Escolha um objeto a sua volta e cite alguns de seus atributos
comportamentos.**



Exercício:

Crie uma classe Pessoa com os atributos nome e idade.

Depois, crie 2 objetos dessa classe e mostre na tela o nome e a idade de cada pessoa.



Exercício:

Crie uma classe Cachorro com os atributos nome e raça.

Adicione um método latir() que imprime a mensagem: “Au Au!

O cachorro {NOME} está latindo.” Crie um objeto da classe e

chame o método.



Exercício:

Crie uma classe Aluno com os atributos nome, nota1 e nota2. Adicione um método que calcule a média e outro que mostre se o aluno foi aprovado (média ≥ 7) ou reprovado. Crie dois alunos e mostre o resultado.



Exercicio:

Crie uma classe ContaBancaria com os atributos titular e saldo. Adicione os métodos depositar(valor) e sacar(valor). Crie um objeto, faça um depósito e um saque, e mostre o saldo final.



Exercicio de fixação

Crie uma classe chamada Animal, com os atributos espécie e som, e o método emitir_som() que simula o som que o animal faz.



Exercicio de fixação

Crie uma classe chamada Animal, com os atributos espécie e som, e o método emitir_som() que simula o som que o animal faz.



Atividade Prática de Laboratório C++ 2 em 15/06/2026

Sistema de Bolão da Copa (com vetores e classes)

Com base no trabalho anterior (classe `NotaFiscal`), você deverá evoluir o sistema para gerenciar um bolão da Copa do Mundo.

O novo programa em C++ deve:

1. Registrar os resultados reais de 4 jogos da Copa utilizando um vetor estático.
2. Registrar os palpites de 3 apostadores (nomes fixos ou digitados) utilizando vetores dinâmicos (`new/delete` ou `vector`).
3. Armazenar os palpites de cada apostador em uma classe `Apostador`, contendo:
 - Nome
 - Pontuação total
 - Um vetor (dinâmico ou estático) com os palpites para cada jogo
4. Comparar palpites com os resultados reais e calcular a pontuação conforme regra de bolão (exemplo: acertar o vencedor = 5 pontos, acertar o placar exato = 10 pontos, etc.).
5. Exibir um ranking final (ordenado do maior para o menor pontuação) com:
 - Nome do apostador
 - Pontuação total



Atividade Prática de Laboratório C++ 2 em 15/06/2026

Requisitos técnicos obrigatórios

- Criar pelo menos duas classes:
 - Jogo (com placar real)
 - Apostador (com nome, palpites e pontuação)
- Usar vetor estático para armazenar os 4 jogos reais.
 - Usar vetor dinâmico (alocado com `new[ ]`) para armazenar os palpites de cada apostador.
 - Destruir corretamente a memória alocada com `delete[ ]` (se usar alocação manual).
 - (Opcional, mas permitido) usar `vector` da STL, desde que explique a diferença entre estático/dinâmico.
- O programa deve ter um menu simples para:
 - Cadastrar resultados dos jogos
 - Cadastrar palpites dos apostadores
 - Calcular pontuação
 - Exibir ranking

```
=== RANKING DO BOLÃO ===  
1º - Ana - 25 pontos  
2º - Bruno - 15 pontos  
3º - Carla - 10 pontos
```



SISTEMA DE BOLÃO DA COPA

C++ com Vetores e Classes



Objetivo: Gerenciar um bolão da Copa do Mundo registrando resultados reais, palpites dos apostadores, calculando a pontuação e exibindo o ranking final!

REQUISITOS OBRIGATÓRIOS

- ✓ Vetor **estático** para os **4 jogos reais**
- ✓ Vetor **dinâmico (new[])** para os palpites dos apostadores
- ✓ Pelo menos **2 classes**: **Jogo** e **Apostador**
- ✓ Calcular **pontuação**: vencedor = **5 pts** | placar exato = **10 pts**
- ✓ Menu simples: **cadastrar, calcular e exibir ranking**

CLASSES



class Jogo

```
+ timeCasa : string  
+ timeFora : string  
+ golsCasa : int  
+ golsFora : int
```

exibir()

class Apostador

```
- nome : string  
- pontuacao : int  
- palpites : Jogo*  
- qtdJogos : int
```

```
+ cadastrarPalpites(Jogo jogos[], int n)  
+ calcularPontuacao(Jogo jogos[], int n)  
+ getNome() : string  
+ getPontuacao() : int
```

VETOR ESTÁTICO – JOGOS REAIS

Jogo jogosReais[4];

	Casa		Fora
Jogo 1	2	x	1
Jogo 2	1	x	1
Jogo 3	3	x	0
Jogo 4	2	x	2

VETOR DINÂMICO – PALPITES DOS APOSTADORES

Apostador* apostadores = new Apostador[3];

Apostador 1

palpites = new Jogo[4]
J1 2 x 0
J2 1 x 1
J3 3 x 1
J4 2 x 2

Apostador 2

palpites = new Jogo[4]
J1 1 x 1
J2 2 x 1
J3 3 x 0
J4 1 x 2

Apostador 3

palpites = new Jogo[4]
J1 2 x 1
J2 1 x 0
J3 2 x 0
J4 2 x 2

REGRAS DE PONTUAÇÃO



Acertar o placar exato = **10 pontos**



Acertar apenas o vencedor (ou empate) = **5 pontos**

FLUXO DO SISTEMA



1. Cadastrar resultados dos jogos



2. Cadastrar palpites dos apostadores



3. Calcular pontuação



4. Exibir ranking final

RANKING FINAL (Exemplo)

POSICÃO	APOSTADOR	PONTUAÇÃO
1º	João	32 pts
2º	Maria	27 pts
3º	Pedro	21 pts



Dinâmico = tamanho pode variar em tempo de execução (new[]) | **Estático** = tamanho fixo definido em tempo de compilação

Atividade Prática de Laboratório C++ 2 em 15/06/2026



=== RANKING DO BOLÃO ===

1º - Ana - 25 pontos

2º - Bruno - 15 pontos

3º - Carla - 10 pontos



- <https://forms.gle/aHcxFZKhSCEChhnr6>